

NumPy, on one card

Arrays are lists that can do maths to themselves. Check .shape, drop the loop, and let np.linalg do the algebra. Session 2, and every session after it.

Making an array

One row of numbers

import numpy as np

```
prices = np.array([650, 820, 500, 1100, 940])
prices.shape      # (5,)   five numbers in a row
prices.dtype      # int64  what kind they are
```

→ Print .shape first, every single time. Nearly every NumPy error is a shape you assumed but never checked.

A table of numbers

```
flats = np.array([[650, 8],          # size, floor
                  [820, 3],
                  [500, 12]])
flats.shape      # (3, 2) -> 3 rows, 2 columns
flats.ndim       # 2      how many dimensions
flats.size       # 6      how many numbers
```

Arrays you did not type out

```
np.zeros(3)      [0. 0. 0.]
np.ones((2, 2))  a 2x2 of ones
np.full(3, 7)    [7 7 7]
np.arange(0, 10, 2)  0 2 4 6 8 (stops before 10)
np.linspace(0, 100, 5)  5 points, evenly spaced, ends included
np.eye(3)        identity matrix
```

! arange stops BEFORE the end, linspace INCLUDES it. This trips everyone up once.

Random numbers you can reproduce

```
rng = np.random.default_rng(0)  # 0 = the seed
rng.normal(0, 4.0, size=30)     # bell curve
rng.integers(1, 7, size=10)     # dice rolls
rng.random(5)                  # 5 numbers in [0, 1)
rng.choice(cities, size=3)      # pick at random
```

→ Same seed, same numbers, every run -- so your results are checkable. In class you will also see the older np.random.seed(0) then np.random.normal(...).

Getting numbers back out

One row: index and slice

```
sizes = np.array([650, 820, 500, 1100, 940])
sizes[0]      # 650   counting starts at 0
sizes[-1]     # 940   last one
sizes[1:3]    # 820, 500  stops before 3
sizes[:2]     # first two
sizes[2:]     # from 2 to the end
```

A table: always use the comma

```
flats[1, :]    # row 1, every column
flats[:, 0]    # every row, column 0
flats[0:2, 0:2] # a 2x2 block
flats[1, 0]    # a single number
```

→ Read [rows, columns]. The comma form is what the course uses -- flats[1][0] also works but reads badly and is slower.

Slices are windows, not copies

```
part = original[1:4]
part[0] = 999      # original changed too!
```

```
safe = original[1:4].copy()  # a real copy
```

! A slice looks at the same memory. Change the slice and you change the original. .copy() when you mean copy.

Pick by a condition

```
big = sizes > 800      # [False True ...] a mask
sizes[big]             # only those values
sizes[sizes > 800]     # same, in one line
big.sum()              # how many are True
np.where(sizes > 800, "big", "small")
```

! Use & | ~ (not and/or/not), and put each test in brackets: sizes[(a > 1) & (b < 5)].

Doing maths to all of it at once

No loop needed

```
prices * 1.08          # every price, +8%
prices + registration  # element by element
prices ** 2
np.sqrt(prices)
np.round(prices, 2)
```

→ This is the whole point of NumPy: one line replaces the for loop, and runs far faster.

Broadcasting: different shapes, still fine

```
flats + np.array([10, 0])  # (3,2) + (2,) -> per column
prices * 2                 # (5,) * scalar
```

→ NumPy stretches the smaller shape to fit, comparing dimensions right to left. Sizes must match or be 1.

! Shapes (3,2) and (3,) do NOT broadcast. Reshape to (3,1) first, or you get a ValueError.

Summarising

BEYOND CLASS

.sum()	total
.mean()	average
.std() .var()	spread
.min() .max()	extremes
.cumsum()	running total
np.median(a)	middle value
np.percentile(a, 95)	the 95th percentile

axis = the direction you collapse

BEYOND CLASS

```
flats.sum()          # one number, everything
flats.sum(axis=0)    # down the rows -> one per COLUMN
flats.sum(axis=1)    # across the columns -> one per ROW
flats.sum(axis=1, keepdims=True)  # keep it (3,1), not (3,)
```

→ axis=0 squashes the rows away, axis=1 squashes the columns away. Check the shape of the answer and you will never guess wrong.

→ keepdims=True is what makes row shares work: flats / flats.sum(axis=1, keepdims=True).

Where is it, not what is it

BEYOND CLASS

a.argmax()	position of the largest
a.argmin()	position of the smallest
np.argsort(a)	positions that would sort it
np.sort(a)	a sorted copy
np.unique(a)	distinct values

Linear algebra -- the S02 headline

Solve A x = b

```
A = np.array([[2.0, 1.0],
              [1.0, 3.0]])
b = np.array([50.0, 50.0])
```

```
prices = np.linalg.solve(A, b)      # the answer
```

```
np.allclose(A @ prices, b)          # True -- always check
```

→ Use solve, never inv(A) @ b. Same answer, but solve is faster and far more accurate.

Matrix moves

A @ B	matrix multiply
A * B	element by element (NOT matrix)
A.T	transpose
np.linalg.det(A)	determinant
np.linalg.inv(A)	inverse (rarely what you want)
np.linalg.matrix_rank(A)	rank

! * is not matrix multiplication. @ is. This is the most expensive typo in the course.

Fit a line to data

```
X = np.column_stack([sizes, np.ones(len(sizes))])
solver_output = np.linalg.lstsq(X, prices, rcond=None)
weights = solver_output[0]
slope, intercept = weights
```

```
slope * new_size + intercept      # predict
```

→ column_stack glues 1-D arrays together as columns -- the standard way to build a design matrix.

Directions that do not turn

```
values, vectors = np.linalg.eig(M)
vectors[:, 0]      # first eigenvector
np.allclose(M @ v, lam * v)      # A v = lambda v
```

```
U, s, Vt = np.linalg.svd(B)      # A = U S V-transpose
```

Reshaping, joining, gotchas

Change the shape, keep the numbers

```
a.reshape(3, 2)      same data, new shape
a.reshape(-1, 1)     one column, -1 = you work it out
a.ravel()            flatten to 1-D
a[:, None]           add an axis -> (n,) to (n,1)
```

Sticking arrays together

```
np.column_stack([a, b])      as columns (taught in S02)
np.vstack([a, b])            stack on top
np.hstack([a, b])            stack side by side
np.concatenate([a, b], axis=0) the general one
```

Missing and infinite

```
np.isnan(a).sum()      # how many gaps
np.nanmean(a)          # mean, ignoring the gaps
a[~np.isnan(a)]        # drop them
! np.nan == np.nan is False. Never test for a gap with ==; use np.isnan().
```

Mistakes everyone makes once

! * is element-wise, @ is matrix multiply.

! An int array silently drops decimals: a = np.array([1,2,3]); a[0] = 1.9 stores 1. Use dtype=float or np.array([1., 2., 3.]).

! Comparing floats with == fails on rounding. Use np.allclose(a, b) or np.isclose(a, b).

! A slice is a view, but a mask makes a COPY. a[a > 5] = 0 works; a[a > 5][0] = 0 changes nothing and does not warn you.

! If a > 5: raises "truth value of an array is ambiguous" -- an array of True/False is not one answer. Say (a > 5).any() or .all().

! np.float, np.int and np.bool were removed in NumPy 1.24, and np.NaN in 2.0. Write float, int, bool, np.float64, np.nan.

Saving your arrays

```
np.save("a.npy", a)      one array
np.load("a.npy")         read it back
np.savetxt("a.csv", a, delimiter=",") as text
df.to_numpy()            a pandas frame -> an array
pd.DataFrame(a)          and back again
```

Maths, applied to every element

```
np.sqrt(a)              square roots
np.exp(a) np.log(a)     e^x and its inverse
np.abs(a)               drop the sign
np.sin(a) np.cos(a)    radians, not degrees
np.clip(a, 0, 100)     force into a range
np.round(a, 2)         2 decimal places
```

→ These all return a NEW array; the original is untouched.

Checking an answer

```
np.allclose(a, b)      equal, allowing for rounding
np.array_equal(a, b)   exactly equal
(a > 0).all()          is EVERY one positive?
(a > 0).any()          is ANY one positive?
np.isfinite(a).all()   no NaN or infinity
```

→ Ending a calculation with an allclose check is the single cheapest habit in this course.

Types, and why they bite

```
a.dtype      # int64 / float64 / bool / <U5
a.astype(float) # convert to a new array
np.array([1, 2, 3], dtype=float)
```

! Whole numbers make an int array. Store 1.9 in it and you get 1, with no warning at all. Ask for floats the moment decimals matter.

Recipes you will reuse

Put columns on a level footing

```
centred      = a - a.mean()
standardised = (a - a.mean()) / a.std()      # mean 0, sd 1
rescaled     = (a - a.min()) / (a.max() - a.min()) # 0 to 1
```

```
# per column of a table, not the whole thing:
(flats - flats.mean(axis=0)) / flats.std(axis=0)
```

→ Every model in Module 2 wants its inputs on comparable scales. This is that step, written out.

How far apart are two points?

```
np.linalg.norm(p - q)      # straight-line distance
np.linalg.norm(a)          # length of a vector
np.linalg.norm(A @ x - b)  # how wrong is this answer
```

→ norm is Pythagoras for any number of dimensions: the square root of the sum of the squares.

Simulate something many times

```
rng = np.random.default_rng(0)
shocks = rng.normal(0.0004, 0.01, size=(1000, 250))
paths = 100 * np.exp(shocks.cumsum(axis=1)) # 1000 price paths
```

```
paths[:, -1].mean()      # average final value
```

```
np.percentile(paths[:, -1], 5) # the bad 5% case
```

→ Build every run at once as rows of a table, then summarise down axis=1. No loop anywhere.

Counting and binning

```
np.bincount(labels)      how many of each 0,1,2...
np.histogram(a, bins=10) counts and bin edges
np.unique(a, return_counts=True) values and their counts
np.digitize(a, edges)    which bin each value falls in
```

Sort a table by one column

```
order = flats[:, 0].argsort() # positions that sort column 0
sorted_flats = flats[order]   # apply to whole rows
flats[flats[:, 0].argsort()[::-1]] # biggest first
```

→ Sort the column, then use those positions to reorder everything -- so the rows stay intact.

When the shapes do not match

ValueError: operands could not be broadcast together with shapes (3,2) (3,)

→ Print both .shape values. Then either transpose one (A.T), or add the missing axis (a[:, None]) so the dimensions line up right to left.